

COMP3702 Artificial Intelligence

Semester 2, 2026

Tutorial 1 - Sample Solutions

Recall that the following components are required to solve an agent design problem:

- **State Space (S)**
The set of all possible configurations of the world the agent is operating in (sometimes called the *set* of states in discrete state systems).
A state is denoted $s \in S$.
- **Action Space (A)**
The set of all possible actions the agent can perform (sometimes called the *action set* in the discrete case).
An action is denoted $a \in A$.
- **World Dynamics/Transition Function ($T : S \times A \rightarrow S'$)**
A function that specifies how the configuration of the world changes when the agent performs actions in it.
A system model.
We sometimes write $T(s, a, s')$ or $P(s'|s, a)$ when the transition function is stochastic.
- **Percept Space (P)**
The set of all possible things the agent can perceive or sense. This is also sometimes called the *observation space*. e.g. for a camera, the set of all possible 128x128 RGB images
In a fully observable system, Percept Space = State Space
- **Perception Function ($Z : S \rightarrow P$)**
A function that maps a world state to a perception.
In a fully observable system, the Percept Function is the Identity Function, $I(s) = s$.
- **Utility Function ($U : S \rightarrow \mathbb{R}$)**
A function that maps a state (or a sequence of states) to a real number, indicating how desirable it is for the agent to occupy that state/sequence of states. We sometimes write $U(s) = x$, some reward value, which we want to maximise. In many situations, we utilise a cost function instead, and aim to minimise its value.

Question 1 - Tic-Tac-Toe

With the above components in mind, an agent to play Tic-Tac-Toe can be designed.

First, we notice that Tic-Tac-Toe is a *fully observable* game, meaning that the agent always knows the exact state of the game from observing the board; there is no hidden information that the agent cannot perceive. This means that the **percept space** is the same as the state space; $P = S$. Following from this, the **perception function** Z simply maps each state to itself (i.e. identity function). Thus, P and Z are not required, and we only need to define A , S , T , and U .

Let us first consider the **action space**, A . The game board is a 3×3 grid, meaning there are 9 possible cells at which the agent can make a mark. If we think about ordering these cells from the top left to the bottom right, they can each be assigned a number from 1 to 9 to indicate their position on the board. Using this representation, $A = \{1, 2, 3, \dots, 9\}$, where, for example, action 1 indicates making a mark at position 1 (the top left corner) of the board.

Using some similar logic, we can formulate a concise representation of the state of the game board. The state can be represented by a vector (or string) of length 9, with each possible element of the vector having 3 legal values, O (for a nought), X (for a cross), or e (for an empty cell). An example state in this form is $(X, e, X, e, O, X, O, X, O)$. The **state space** is then just all possible legal vectors of this form. Note that a legal vector must contain at most 5 noughts or 5 crosses because of the turn-based nature of the game. Legal vectors also cannot contain a winning configuration of both noughts and crosses, as if a winning configuration is obtained by either player, the game ends.

You can visualise the **transition function** T as a lookup table with a row for each $(state, action)$ pair that stores the state that is the result of performing $action$ in $state$. This function encapsulates both our move, and our opponent's move. Because the game is non-deterministic (we don't know what our opponent is going to do), the result of performing $action$ in $state$ could lead to multiple possible new states.

There are many possible utility functions for this game, and the choice of **utility function**, U , will determine how the agent behaves. One simple utility function is to assign a value of 10 to any state where the agent has a winning configuration of noughts/crosses, -10 to any state where the opponent has a winning configuration, and 0 for any other state. This utility function will reward the agent for winning the game, however it does not take into account the number of actions used to reach a winning state. Since the goal of the agent is to win in the least number of actions possible, one could assign a utility of -1 to any state that does not contain a winning configuration for the agent or the opponent. This way, the more actions the agent takes to win, the less utility the sequence of states it passes through has.

Question 2 - Tic-Tac-Toe Valid States and ChatGPT

Note that the code generated by ChatGPT considers the full game tree! If you consider the upper limit for the size of the state space, it would be all arrangements of X, O and empty, which is $3^9 = 19,683$. This count includes many invalid positions e.g. states where the number of noughts and crosses are not within 1 of each other, and states where both players are in a winning state, as well as states that are impossible to reach as one of the agents would have already won. If we exclude the invalid states, there remain 5478 states of tic-tac-toe.

Question 3 - Intelligent Air-Conditioner

The components of an intelligent robot vacuum cleaner agent could be as follows:

- **abilities:** suction on/off, turn to a direction $0^\circ \leq \text{heading} < 360^\circ$, move forward/back.
- **goals:** clean area in fastest time possible
- **prior knowledge:** map of the environment
- **stimuli/percepts:** radar/lidar, RGB imagery
- **past experiences:** previous paths traversed for the map and their time taken

Another representation could be as follows, with a discretised robot vacuum cleaner in a two-cell world, as provided by Russell & Norvig:

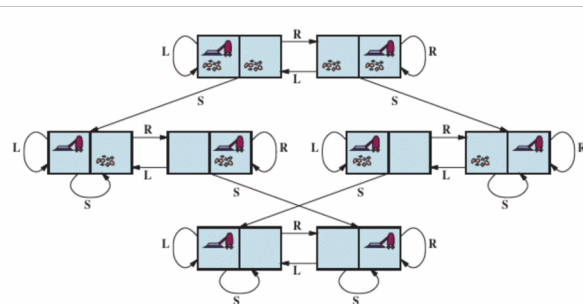


Figure 3.2 The state-space graph for the two-cell vacuum world. There are 8 states and three actions for each state: L = Left, R = Right, S = Suck.

- **abilities/action space:** Move to other cell, suck.
- **goals:** Have all cells of the floor clean
- **prior knowledge:** Sucking a dirty floor cell will clean up the cell
- **stimuli/percepts:** Visual (ability to observe the floor directly under it)
- **past experiences:** Previous visits to the cells and whether they were clean or not

Question 4 - Percept Space vs State Space for Robot Vacuum

For this agent, the **state** could consist of three components:

- the robot's two-dimensional location in a two-dimensional map
- the robot's orientation
- what regions have been cleaned so far in this map

The **state space** would be all legal combinations of the robot's location, orientation and regions cleaned.

The **percept space** comes from the agent's sensors and what they can sense or perceive. A robot vacuum cleaner might have a number of sensors to help perceive its environment. e.g. laser sensors, cameras, bump/pressure sensors, cliff sensors, wheel rotation sensors to measure speed, pressure-sensitive height sensors to avoid going underneath low furniture, etc. For a robot vacuum cleaner with laser sensors, the **percepts** would be the distance to potential obstacles including objects and walls. The **percept space** would then be all possible values of perceived distances to obstacles (usually several rays in its field of view) as in Figure 1.

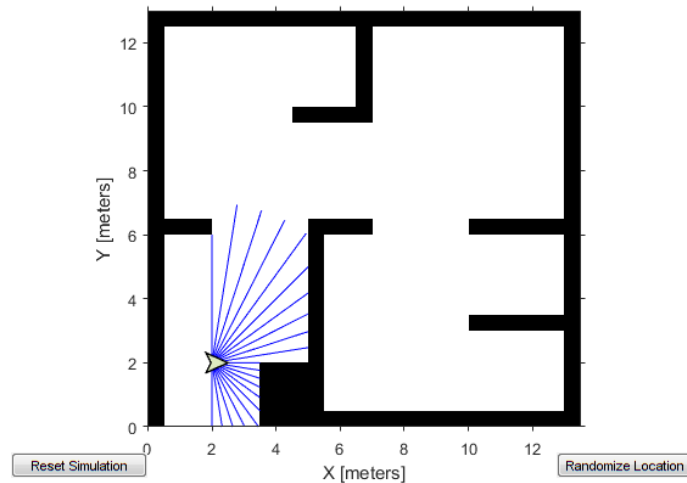


Figure 1: Laser sensor example.

Source: <https://www.mathworks.com/help/nav/ug/path-following-with-obstacle-avoidance-in-simulink.html>

From a single time instant, it is not possible for the robot to exactly locate its position in the environment because there is not a one-to-one mapping from what it can perceive (its percepts or observations) to its location, i.e. the problem is not fully-observable, only partially observable. From a sequence of percepts, it could estimate its location and orientation in the environment, and by maintaining the history of its location while cleaning, it could determine what areas have already been cleaned to estimate the current state.

An example of robot localisation using sonar and a technique called Particle Filtering can be seen here: <https://rse-lab.cs.washington.edu/projects/mcl/animations/global-floor.gif>

Question 5 - Navigation App

Agent Design Components:

- State Space:
 - Set of all valid street addresses?
 - Road intersections?
 - Set of all landmarks (e.g., buildings at a university)
 - $\{(x, y) \mid x, y \in \mathbb{R}\}$? Discretisation?
- Action Space
 - Set of all navigation directions that can be given, e.g.:
 - {continue straight, turn left, turn right, etc.}
 - $\{(\text{distance}, \text{heading}) \mid 0 \leq \text{distance} < \text{max_dist}, 0^\circ \leq \text{heading} < 360^\circ\}$
- World Dynamics:
 - Position changes to the next node in the direction the agent selected
- Utility Function:
 - Reach the goal state with minimum distance traveled

- Utility = $-1 \times$ distance travelled
- Percept Function and Percept Space (Note that the question specifies that the location provided by the GPS is correct, i.e. the percept exactly reveals the agent’s state, so this is a **fully observable** problem)
 - Percept Space = GPS location
 - Percept Function = the Identity function, OR a lookup table / function converting GPS to whatever the state representation is (e.g. intersection number, quantised latitude and longitude)

A more detailed answer below:

There are many ways to answer this question, as it depends on the type of navigation app and what kinds of factors we want to be able to handle. Here we consider the case of a navigation app specifically for finding the shortest (not fastest) path to take in a network of roads. We ignore other ‘adversarial’ agents, such as people and cars in the environment, and don’t include these entities as part of the states of the environment, mainly because the potentially random nature of these entities is far too difficult to model. To add a little bit of extra realism, we will also model one key feature of a road network, which is the fact that at an intersection some turns might be illegal! For this reason, we can’t simply model states as intersections between roads, as the road we take on the way into the intersection determines which roads we can take on the way out.

Looking at the effects of the 3 assumptions listed: The first assumption will influence the formulation of the utility function for the agent, as the utility function dictates how the agent behaves. The second assumption causes the environment the agent is operating in to be *deterministic*, as the agent knows exactly where everything is in the environment. The third assumption causes the environment to be *fully observable* because the GPS reading tells the agent its exact location in the world.

a) Agent Design

As the environment is fully observable, defining the percept space and the perception function is not required. So, like Question 1, only A , S , T , and U need to be defined.

State space: The states of this problem describe locations of the user’s vehicle in the world; in principle, these locations might be described by GPS coordinates. To simplify the problem, we consider states as corresponding to segments of road between junctions, where a “junction” is simply a point at which roads meet and thus it is possible to go in at least two different directions (e.g. go forward, turn left, take an exit off a highway, or take a specific exit out of a roundabout).

Additionally, since the dynamics of the world are very different depending on which direction one is travelling along a road, the direction of travel must also be included. Thus the state space can be formally described as

$$S = \{r : r \text{ is a road} \times \{1, 2, 3, \dots\} \times \{increasing, decreasing\}\},$$

where *increasing* and *decreasing* refer to whether house numbers increase or decrease in the direction of movement. This definition means that each state is a 3-element tuple consisting of a road, segment number, and the direction of travel such as

$$S = (\text{Gympie Rd}, 1, \text{increasing}).$$

Note that not all states in the state space will be valid; some will be illegal or unreachable, since some roads are one-way roads, and each road only has finitely many segments.

Action space: Note that the app doesn't control the car, the app's user does! Thus the app's actions are not to drive the vehicle, but to give instructions to the user; the actions look like "tell the user to turn left" and not "turn left". This could be done in many ways, but the essential question is this: at a junction, which way out should the user take?

One way to represent this information within our agent is to number the possible ways out of a junction in clockwise order, so the action space is:

$$A = \{1, 2, 3, \dots\}$$

To make the navigation app more user friendly, hopefully a translation layer would be added so that "1" would be translated into, say, "turn left" or "take the first exit" depending on what kind of junction is coming up.

World dynamics/Transition function: The world dynamics describe how the environment reacts in response to the agent's actions. To make the problem easier, we assume that the user of the app will do what they are instructed to do. The world dynamics tell us what the next state will be, given the current state and the action the agent takes; this is a function¹:

$$T : S \times A \rightarrow S',$$

which can be represented as a table with entries like this:

State (S)	Action (A)	Next state (S')
(Gympie Rd, 40, increasing)	1	(Albany Creek Rd, 1, increasing)
(Gympie Rd, 40, increasing)	2	(Gympie Rd, 41, increasing)
(Gympie Rd, 40, increasing)	3	(Robinson Rd W, 20 decreasing)

The table above would imply that, for example,

$$T((\text{Gympie Rd}, 40, \text{increasing}), 1) = (\text{Albany Creek Rd}, 1, \text{increasing})$$

when represented in function form.

Percept space: Based on assumption (3), the problem is fully observable, and thus we do not need to formally specify the percept space. Equivalently, we could say that

$$P = S$$

Percept function: Since the problem is fully observable, a percept function is not required. Equivalently, it can be described by the identity map, i.e.,

$$Z = S \rightarrow P$$

¹Note: since not all states in the state space are valid, and not all actions are available in each state, it would be more accurate to say the domain of T is a subset of $S \times A$ which represents all legal state-action pairs.

Utility function: Since the agent is supposed to find shortest paths, we can represent this in the form of utility by assigning the agent disutility (i.e. negative utility) proportional to the lengths of the road segments that it travels. This can be represented by a function

$$c : S \rightarrow \mathbb{R},$$

which specifies the length of each segment of road as a real number, e.g. in metres. The agent can then associate its total utility with the total cost of a sequence of states, i.e.

$$U((s_1, s_2, s_3, \dots, s_n)) = - \sum_{i=1}^n c(s_i).$$

b) Defining environment

As already discussed, the assumptions given cause the agent to operate in a deterministic and fully observable environment.

If representing the state as GPS locations consisting of longitude and latitude (which can be expressed as arbitrary precision real numbers), the environment would be continuous. If we choose to represent the state as a triplet of road, segment number and direction, by our design, the state would be discrete.

Because adversarial agents in the environment are not being modelled as part of the state, the environment is also static; the state cannot change while the agent is selecting an action.

Environment Type:

- Discrete vs Continuous
 - Depends on assumptions and representation
 - e.g., street addresses are discrete, latitude and longitude may be continuous (if not discretised)
- Deterministic vs Non Deterministic
 - Deterministic if driver/user always follows direction correctly
 - Non deterministic if driver/user makes a mistake or chooses not to follow directions
- Fully vs Partially Observable
 - GPS fully reveals the state (assuming adequate precision)
 - Information about traffic and pedestrians is not observed
- Static vs Dynamic
 - Map is 100% up to date, so connectivity (e.g., road connections) do not change during an episode $\rightarrow$ static
 - Traffic, pedestrians, etc. may change during an episode, so environment becomes dynamic if these are considered

c) Defining search problem

The translation to a search problem is fairly straightforward. The map can be represented as a graph, where vertices are states and edges are actions, and following an edge from one vertex to another models the transition function.

Similarly to how we thought about the utility function in the agent design problem, we could assign each edge a weight or cost to indicate how ‘expensive’ it is to transition along that edge. This cost could be based on the physical distance between the vertices’ locations on the map (as the goal was to find the shortest path). The search problem is then, given a starting vertex and a goal vertex, find the shortest path (path of least cost) between the two vertices in the graph.

Search Problem:

- **Initial State:** User’s current location
- **Goal State:** User’s desired destination
- **State Graph Representation:** (different options depending on state representation)
 - Landmarks are vertices, paths between landmarks are edges
 - Street addresses and intersections are vertices, road sections are edges, street addresses always have 1 or 2 connections, intersections may have more
 - Graph with grid structure, where each node has a latitude and longitude corresponding to row and column in the grid

Question 6 - Poker Bot

- Action Space
 - {Check, Call, Raise, Fold}
 - Assumes players always raise by the same amount – otherwise 1 action for each raising amount
- Percept Space - The agent’s 2 cards and the cards shown in each stage
- State Space - It can get complicated
 - All combinations of cards for each player including shown cards
 - All combinations of current and previous bets from all players
 - All combinations of total chips for each player and in pot
- Transition Function - Given current state S of (cards,chips,bets..etc), perform action (Check, Call, Raise, Fold), to arrive to a new state S' , where everyone folds and you win all the chips.
 - Number of chips in pot changes based on player bets
 - Chips are awarded to the winner of the round
 - Cards are dealt again at the end of the round/start of next round
 - Other player’s actions
- Utility Function

- +X for winning where X is the total pot
- +5 for an opponent that folds
- +10 for hitting a card that improves your hand

Complications when there are multiple opponents:

- Size of state grows exponentially as number of players increases
- Modelling opponent behaviours becomes very complicated